

Directus AI + MCP — Complete Client Documentation

Project: BoTG — Directus CMS

Prepared by: Encircle Technologies

Date: June 2026

Version: 1.0

Table of Contents

1. [Environment Overview — Current Status](#)
 2. [What is MCP and How it Connects to Directus](#)
 3. [Enabling MCP on Your Directus Instance](#)
 4. [Connecting an AI Client \(Claude\)](#)
 5. [What IS Possible via MCP — Verified Capabilities](#)
 6. [Our AI Workflow in Practice](#)
 7. [Creating Collections, Fields, Translations & Flows via AI](#)
 8. [Custom Extensions — How They Work with AI](#)
 9. [Critical Considerations Before Any Change — Directus Is a CMS, Not a Freeform App](#)
 10. [Migration Management — Environments & Deployments](#)
 11. [Collaboration, Content Safety & Deployment — Policy and Assessment](#)
 12. [What AI Cannot Do — Limitations](#)
 13. [Summary Capability Table](#)
 14. [Security Best Practices](#)
-

1. Environment Overview — Current Status

1.1 BoTG Environments

Environment	URL	Directus Version	MCP Supported
Production	https://content.botg.cloud/admin/login	11.17.4	 Yes
Staging	https://staging.content.botg.cloud/admin/login	11.17.4	 Yes
Development	https://dev.content.botg.cloud/admin/login	11.17.4	 Yes

All BoTG environments are running Directus **11.17.4**, which is above the minimum required version for the built-in MCP server. MCP can be enabled on all three environments immediately.

1.2 MCP Minimum Version Requirement

The built-in MCP server was introduced in Directus **v11.12.0**. Any instance below this version cannot use the native MCP functionality. All BoTG environments (11.17.4) already meet this requirement.



2. What is MCP and How it Connects to Directus

2.1 What is MCP?

MCP (Model Context Protocol) is an open standard developed by Anthropic that allows AI assistants like Claude to connect directly to external tools and systems — not just read about them, but actually interact with them in real time.

Think of it as a **structured API bridge** between the AI and your Directus instance:

```
Claude (AI) ← MCP Protocol → Directus MCP Server ↔ Directus Database
```

Without MCP, you would have to:

- Copy-paste data from Directus into the chat
- Describe your schema manually to the AI
- Ask the AI to generate code you then execute yourself

With MCP, Claude can:

- Query your live database directly
- Read schema structures automatically without any manual description
- Create, update, and manage content
- Build and modify automation flows
- Trigger existing workflows on your behalf

2.2 How MCP Connects to Directus

The Directus MCP server exposes a set of **tools** (functions) that Claude can call. The connection flow is:

1. Claude Code / Claude Desktop starts
2. Reads MCP configuration (settings.json or inline config)
3. Connects to the Directus MCP server endpoint
4. MCP server authenticates to Directus using your Static Token or OAuth
5. Claude can now call MCP tools as part of any conversation

The MCP server we use is the **official Directus MCP package** ([@directus/mcp](#)), either via the built-in server (Directus 11.12+) or the standalone npm package for older approaches.

2.3 Tool Naming Convention

Once connected, Claude gains access to a set of named tools. The prefix is the name you give the connection in your configuration. For example, if you name the connection **directus-staging**, all tools are prefixed:

- **mcp_directus-staging__schema**
- **mcp_directus-staging__items**

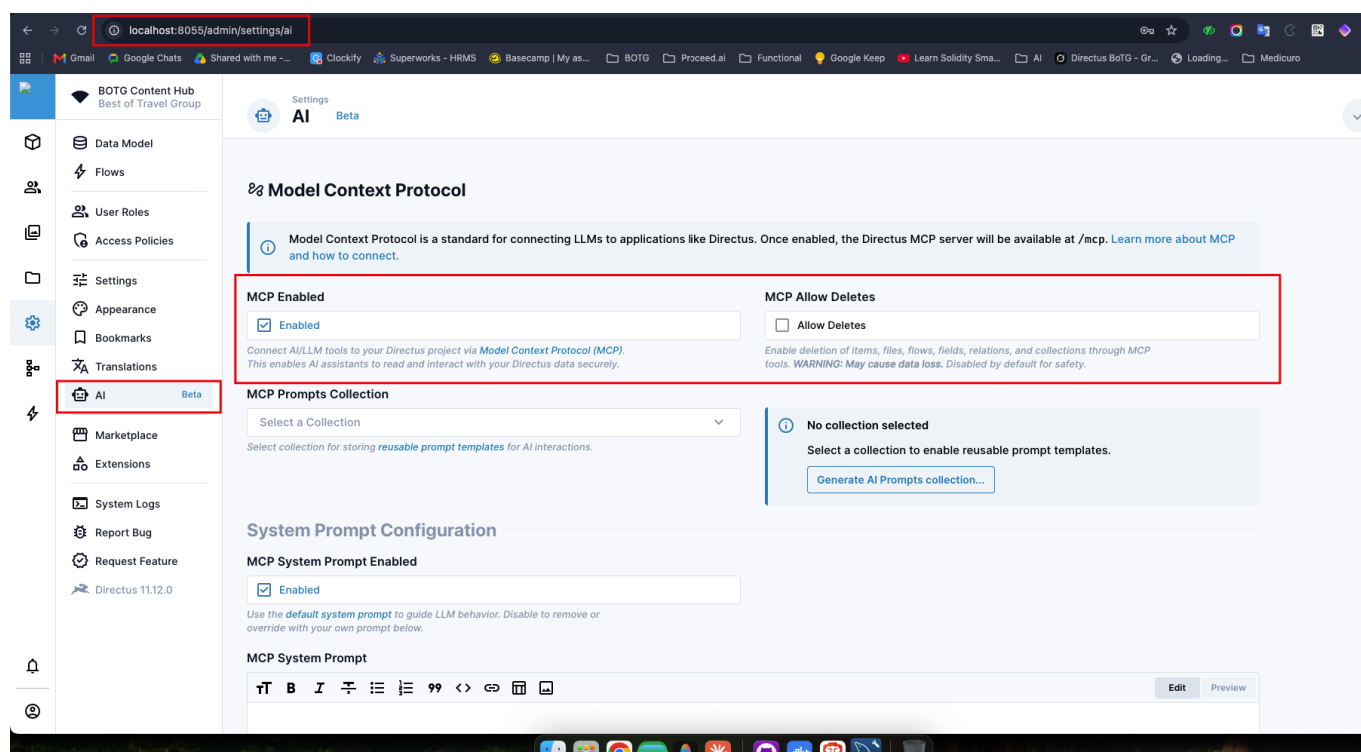
- `mcp__directus-staging__collections`
- `mcp__directus-staging__fields`
- `mcp__directus-staging__flows`
- `mcp__directus-staging__trigger-flow`
- (and more — see Section 5)

2.4 Quick Reference — Enabling MCP and Connecting a Client

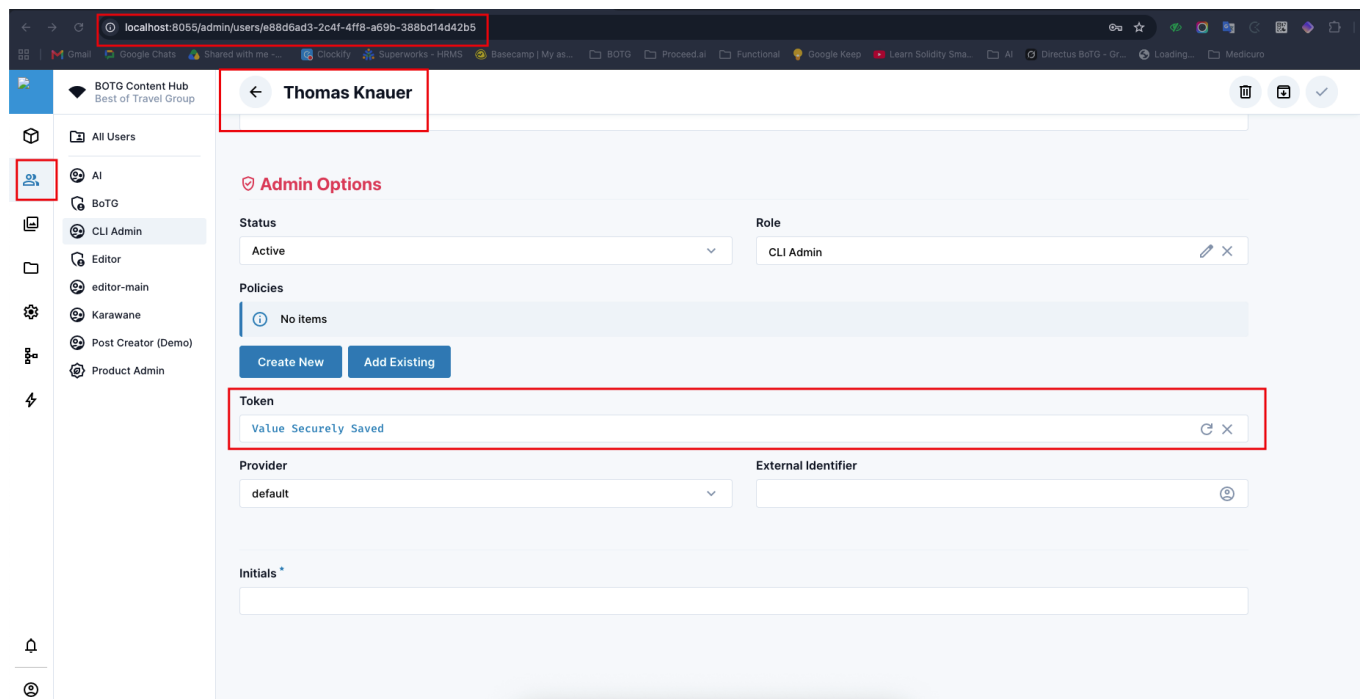
This is the short end-to-end path. Detailed instructions follow in Sections 3 and 4.

Enabling MCP — for each environment where MCP should be available:

1. Navigate to `/admin/settings/ai`
2. Enable the MCP server (toggle **MCP Enabled**)
3. Open the desired user account (e.g. *Thomas Knauer*)
4. Generate an MCP access token for that user
5. Copy and securely store the generated token — it will be required by the AI client



Directus → Settings → AI → Model Context Protocol. The "MCP Allow Deletes" option is disabled by default — it must be explicitly enabled to permit deletion of items, files, flows, fields, relations, and collections through MCP tools.



Open the user account (Settings → Users), generate a token in the **Token** field, then save. The value is shown only once at generation time and afterward displays as "Value Securely Saved".

Connecting an AI Client

Directus provides setup instructions for various AI clients:

- <https://directus.com/docs/guides/ai/mcp#supported-clients>
- <https://directus.com/docs/guides/ai/mcp/installation#connect-your-ai-client>

For example, with **Claude Code**:

OAuth (Recommended)

```
claude mcp add --transport http directus https://your-directus-url.com/mcp
```

Then start Claude Code and complete the browser authorization flow.

Static Access Token (if OAuth is not available)

```
claude mcp add --transport http directus https://your-directus-url.com/mcp \
  --header "Authorization: Bearer your-generated-token"
```

Once configured, the AI client will be able to interact with the Directus instance through MCP using the **permissions of the user associated with the generated token**.

If you would like us to assist with the MCP configuration, please let us know.

3. Enabling MCP on Your Directus Instance

3.1 Step-by-Step: Enable MCP in the Directus Admin Panel

1. Log into your Directus Admin panel
2. Navigate to **Settings** (gear icon in the sidebar)
3. Select **AI** from the left menu
4. Scroll down to the **Model Context Protocol** section
5. Toggle **Enable MCP** to ON
6. Click **Save**

Safety setting — MCP Allow Deletes: On the same screen there is a separate **MCP Allow Deletes** toggle, which is **disabled by default**. While disabled, MCP tools cannot delete items, files, flows, fields, relations, or collections — they can only read, create, and update. We recommend leaving this OFF on Production and enabling it only in Staging/Development when a deletion task is genuinely required, then disabling it again. This is the primary guardrail against accidental data loss via AI.

There is also an **MCP System Prompt** section where a default (or custom) system prompt can be configured to guide the AI's behavior, and an optional **MCP Prompts Collection** for storing reusable prompt templates.

3.2 Generate a Directus MCP Access Token

The MCP server authenticates using a **Static Token** — a permanent, non-expiring credential tied to a specific Directus user account.

Step-by-Step:

1. Log into your Directus Admin panel
2. Navigate to **Settings → Users**
3. Open the user account you want the AI to act as
 - **Best practice:** Create a dedicated "AI Agent" user (e.g. `ai-agent@company.com`) rather than using a personal account — this creates a clear audit trail
4. Scroll to the **Token** section
5. Click **Generate Token** (or type a custom value)
6. **Copy the token** — it is only shown once
7. Click **Save**

Important: Store the token securely. Never commit it to Git. Use environment variables or a secrets manager.

3.3 Token Per Environment

Use a **different token for each environment** (development, staging, production). This ensures that a staging breach does not affect production, and that you can revoke access per environment independently.

4. Connecting an AI Client (Claude)

4.1 Method 1 — OAuth (Recommended)

For Claude Code, run this command and complete the browser authorization flow:

```
claude mcp add --transport http directus https://your-directus-url.com/mcp
```

Then start Claude Code and you will be redirected to authorize via the Directus admin login.

4.2 Method 2 — Static Access Token

If OAuth is not available or you prefer a static configuration:

```
claude mcp add --transport http directus https://your-directus-url.com/mcp \
  --header "Authorization: Bearer your-generated-token"
```

4.3 Method 3 — Configuration File (Claude Code mcp_settings)

In your project's `.claude/settings.json` (or `~/.claude/settings.json` for global configuration), add an `mcpServers` block:

```
{
  "mcpServers": {
    "directus-staging": {
      "command": "npx",
      "args": ["-y", "@directus/mcp"],
      "env": {
        "DIRECTUS_URL": "https://your-directus-instance.com",
        "DIRECTUS_TOKEN": "your-static-token-here"
      }
    }
  }
}
```

The key (`"directus-staging"`) becomes the prefix for all tool names.

4.4 Connecting Multiple Environments Simultaneously

You can connect multiple Directus instances at the same time — one per environment. Claude will have tools for all connected instances:

```
{
  "mcpServers": {
```

```
"directus-staging": {
  "command": "npx",
  "args": ["-y", "@directus/mcp"],
  "env": {
    "DIRECTUS_URL": "https://staging.content.botg.cloud",
    "DIRECTUS_TOKEN": "staging-token-here"
  }
},
"directus-production": {
  "command": "npx",
  "args": ["-y", "@directus/mcp"],
  "env": {
    "DIRECTUS_URL": "https://content.botg.cloud",
    "DIRECTUS_TOKEN": "production-token-here"
  }
}
}
```

4.5 Supported AI Clients

Directus MCP works with any MCP-compatible AI client. Confirmed supported clients include:

- **Claude Code** (CLI — primary tool used in this project)
- **Claude Desktop** (Mac/Windows app)
- Any MCP-compatible client that supports HTTP transport

Reference documentation:

- <https://directus.com/docs/guides/ai/mcp#supported-clients>
- <https://directus.com/docs/guides/ai/mcp/installation#connect-your-ai-client>

5. What IS Possible via MCP — Verified Capabilities

These are the **actual tools** available in our **directus-staging** MCP connection, verified against the live instance. Our staging instance has **170+ collections** including **hotels**, **tours**, **cruises**, **campers**, **roundtrips**, **daytrips**, and all their related sub-collections.

5.1 Schema Discovery

The AI can fully explore and understand the database structure **without any manual explanation**.

What it does:

- Lists all collections and collection folders
- Returns field types, validation rules, notes, and display interfaces
- Reads all relationship maps: M2O, O2M, M2M, M2A, and Translation junctions
- Understands field groups, accordions, and alias field configuration

Example prompt:

"What fields does the **hotels** collection have?" → AI calls the schema tool, reads all field definitions, and answers accurately without guessing.

5.2 Data Operations — Items (Full CRUD)

Full Create, Read, Update, Delete on any collection the token has access to.

Operation	Capability
Read	Filter, sort, paginate, search, aggregate, deep-query nested relations
Create	Single or batch, with nested relation creation
Update	Single or batch, partial updates
Delete	By primary key(s)

Supported filter operators: **_eq**, **_neq**, **_in**, **_nin**, **_icontains**, **_starts_with**, **_between**, **_lt**, **_gt**, **_null**, **_nonnull**, **_some** (O2M), **_none** (O2M), **_and**, **_or**

Deep queries: Can filter and sort nested relationships in a single call.

Aggregation: **count**, **sum**, **avg**, **min**, **max** — with **groupBy** support.

Example prompts:

- "Show me all hotels in Germany with status = published"
- "Create a new daytrip with translations for DE, NL, CH"
- "Update the sell_price on all batch_hotel items for locale DE"
- "Count how many tours_prices records exist per destination"

5.3 Collections Management

The AI can design and create the entire database structure.

- Create new collections with full metadata (icon, color, note, sort field, archive config, display templates)
- Configure singleton collections (for settings/global records)
- Set up soft-delete (archive) behavior
- Enable content versioning
- Create collection folders for UI grouping
- Update or delete collections

Example: "Create a `blog_posts` collection with UUID primary key, title, slug, content, published_at, and a status field with archive support."

5.4 Fields Management

Full field CRUD with complete interface and display configuration.

All field types supported:

- Text: `string`, `text`, `uuid`, `hash`
- Numeric: `integer`, `bigInteger`, `float`, `decimal`
- Date/Time: `timestamp`, `datetime`, `date`, `time`
- Boolean, JSON, CSV
- Geospatial: `point`, `lineString`, `polygon`
- Alias (virtual): for relationship display, groups, accordions, flow triggers

Full interface configuration — the AI sets the correct Directus interface for each field (e.g. `input-rich-text-md`, `select-dropdown-m2o`, `list-o2m`, `map`, `input-multiline`, `collection-item-dropdown`) and all display options including notes, conditions, validation, and readonly flags.

5.5 Relations Management

The AI can design and create all Directus relationship types:

Type	Description	Example in this project
M2O	Many-to-One	<code>hotels</code> → <code>hotel_group</code> , <code>daytrips</code> → <code>destination</code>
O2M	One-to-Many	<code>hotel</code> → <code>room_categories</code> , <code>daytrip</code> → <code>surcharges_items</code>
M2M	Many-to-Many (junction)	<code>hotels</code> ↔ <code>activities</code> , <code>daytrips</code> ↔ <code>countries</code>
M2A	Many-to-Any (polymorphic)	Content blocks pointing to multiple types
Translations	Built-in i18n junction	<code>hotels_translations</code> , <code>daytrips_translations</code>

5.6 Files & Media

Files tool:

- Read file metadata (title, type, filesize, dimensions, folder, upload date, tags)
- Filter files by type, folder, size, date
- Update metadata in bulk (titles, tags, descriptions, focal points)
- Import files from external URLs directly into Directus
- Delete files by ID

Assets tool:

- Retrieve actual file content as base64 (images and audio)
- Enables the AI to **visually analyze images** stored in Directus
- Useful for: checking image quality, reading text in images, validating content

Folders tool:

- Create, read, update, delete virtual folders
 - Build folder hierarchy for media organization
-

5.7 Automation Flows

The AI can fully read, create, and modify the automation system.

Our staging instance has **60+ active flows** covering:

- Price calculation and synchronization
- AI-powered translation triggers
- Image badge expiry checks (scheduled daily at 08:00)
- IPTC metadata auto-fill on file upload
- Video metadata auto-fill on upload
- Margin preset application across products
- Publication date monitoring and editor notifications
- MAT (Mobility Advice Text) sync across product translations

Flow trigger types the AI can work with:

Trigger	Description
event	Fires on database events (create/update/delete on specific collections)
schedule	Cron-based scheduling (daily, weekly, etc.)
webhook	External HTTP trigger
manual	Button in the Directus UI or triggered via API
operation	Sub-flow called by another flow

Operation types available:

- **condition** — branching logic
 - **exec** — run arbitrary JavaScript (Node.js)
 - **request** — HTTP requests to external APIs
 - **log** — debug logging
 - **send-email** — email notifications
 - **send-notification** — in-app Directus notifications
 - Read/write/update/delete items within a flow
-

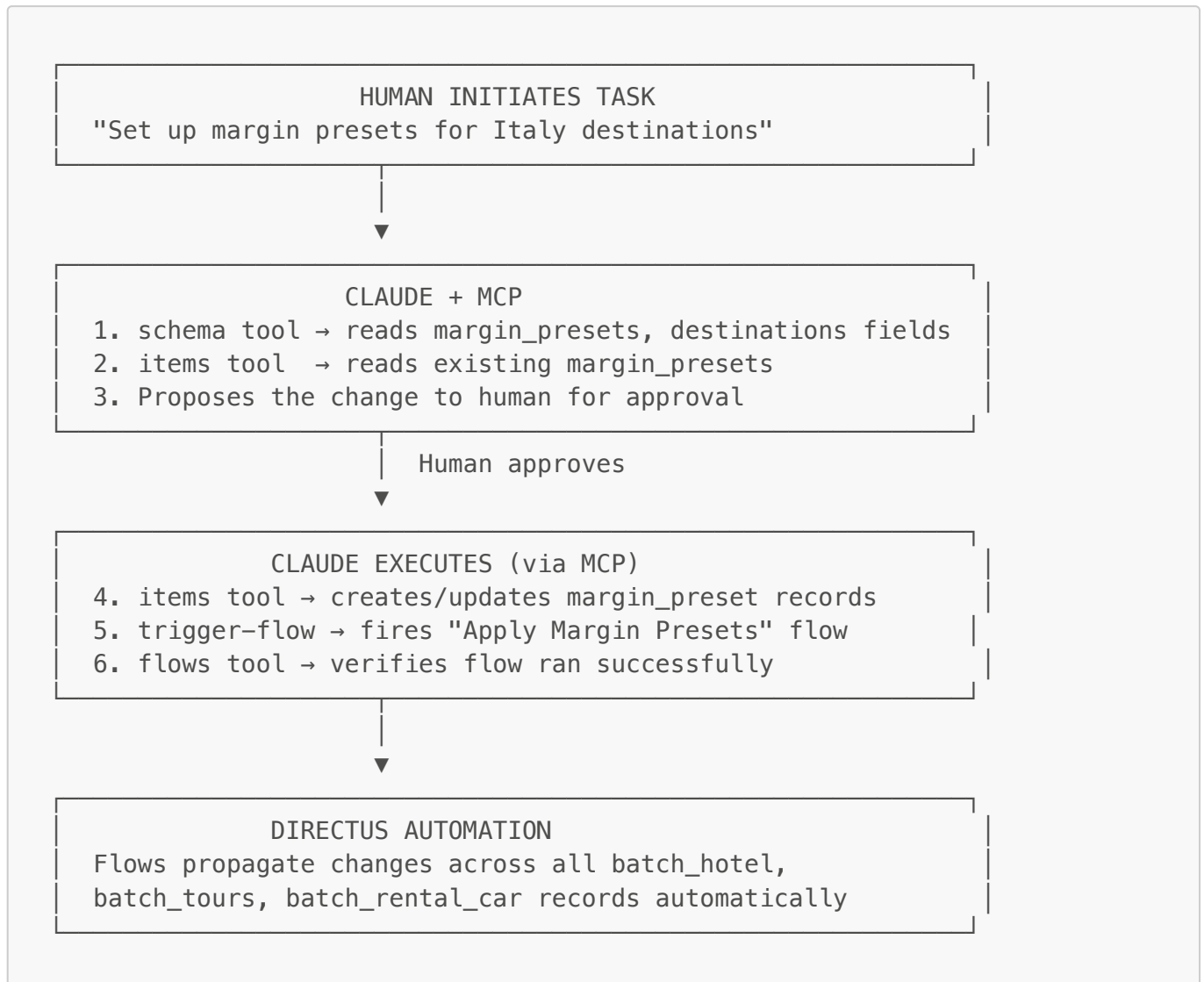
5.8 Triggering Flows Programmatically

The AI can execute **manual flows** directly, passing data payloads and collection context.

Example: Trigger the AI Translate flow on a selection of hotel IDs, or trigger the margin preset application flow to recalculate sell prices for all batch products in one call.

6. Our AI Workflow in Practice

6.1 Standard Workflow — Human Stays in Control



Core principle: AI always reads the schema before writing anything. Every significant change is proposed to the human before execution. Destructive operations (deletes, bulk updates) are always confirmed first.

6.2 Real Examples from This Project

IPTC Metadata Auto-Fill Flow triggers on `files.upload`. AI reads IPTC data embedded in the uploaded image and automatically writes photographer, copyright, keywords, and alt text to the file record — eliminating manual metadata entry.

Video Metadata Auto-Fill Same as IPTC but for video files, which Directus does not natively parse. The flow extracts duration, codec, and resolution metadata.

AI Translations (directus-extension-ai-translations) The "AI Translations" custom interface adds an **AI Translate** button to the standard translations view. When an editor clicks it on a hotel or daytrip record, Claude batch-translates all selected fields from the source language (e.g. de-DE) to all target languages (de-CH, nl-NL) in a single operation. Translations appear as previews highlighted in purple — the editor reviews them, then clicks **Apply All** to accept or **Cancel** to discard.

Price Synchronization Flows A chain of flows ([PRICE CALCULATOR]) automatically regenerates `room_prices`, `cruise_prices`, and `daytrips_prices` whenever a category, occupancy, or price date is created or updated. The AI assisted in designing and replicating this logic across all product types.

Image Badge Expiry A scheduled flow runs at 08:00 daily. It checks expiry dates on image badges across hotels, cruises, and daytrips, and sends in-app notifications to editors before items are automatically unpublished.

MAT (Mobility Advice Text) Sync A single-record `mobility_advice_text` collection holds global per-language mobility advice text. Three flow chains (CREATE, UPDATE, TRIGGER) automatically copy this text into every product's per-language translation rows whenever a product is created or updated, or whenever the MAT record itself is edited. This covers hotels, cruises, tours, and daytrips.

Tours → Daytrips Migration A phased 10-step migration (ongoing) in which AI via MCP created all new `daytrips_*` collections, fields, relations, and flows — all without touching the existing `tours_*` collections. This additive approach allowed the new structure to be verified in staging before the old structure is cleaned up.

7. Creating Collections, Fields, Translations & Flows via AI

This section describes how we use Claude + MCP to build Directus schema from scratch. These are the exact practices followed in this project.

7.1 Naming Conventions

All schema work follows strict naming conventions:

Element	Convention	Example
Collections	Plural, snake_case	hotels, daytrips_translations
Fields	Singular if one, plural if many; snake_case	destination, countries, booking_partner
Junction tables	{collection_a}_{collection_b}	daytrips_directus_files
Translation tables	{collection}_translations	hotels_translations
UI Labels	Title Case for nouns, lowercase for articles	"Hotel Classification", "of", "and"
Content languages	de-DE (primary), de-CH, nl-NL	No en-GB in content fields

7.2 Creating a Collection

Prompt pattern:

```
"Create a daytrips collection with UUID primary key, status field with archive support, sort field, user_created/date_created/user_updated/date_updated audit fields."
```

Claude will:

1. Read existing similar collections for reference (e.g. **hotels**)
2. Create the collection with the correct meta configuration
3. Add all specified fields with correct types and interfaces
4. Confirm the creation and show a summary

What gets configured per collection:

- Primary key type (UUID or integer)
- Sort field for manual reordering
- Status field with **archive_field**, **archive_value**, **unarchive_value**
- Display template (what appears in relation dropdowns)
- Icon and color for the UI sidebar
- Collection folder for grouping

7.3 Adding Fields

Prompt pattern:

"Add a `destination` field to `daytrips` — integer, M2O to destinations collection, required, note 'Primary destination for this daytrip'."

Claude will:

- 1. Read the schema of the target collection and the related collection
- 2. Create the field with correct type, interface (`select-dropdown-m2o`), and display
- 3. Create the relation automatically
- 4. Apply the field note

Common field creation examples from this project:

Field Type	Example
String input	<code>name</code> , <code>slug</code> , <code>booking_email</code>
Text area	<code>internal_remarks</code> , <code>mobility_advice_text</code> (readonly)
Integer	<code>participants_min</code> , <code>children_free_age</code>
Decimal	<code>buy_price</code> , <code>margin_percentage</code>
Boolean	<code>price_start</code> , <code>operator</code>
Date	<code>image_badge_start_date</code> , <code>publish_start</code>
JSON	<code>daytrips_specials</code> , <code>description_supplementary</code>
M2O relation	<code>destination</code> → <code>destinations</code> , <code>season</code> → <code>seasons</code>
Radio	<code>partner_type</code> (values: all/selected), <code>buy_price_type</code> (per_unit/per_person)
Readonly	Any field with <code>options.disabled = true</code> and a condition to show on edit

7.4 Setting Up Translations (i18n)

Directus translations follow a junction-table pattern. For a `daytrips` collection with translatable content:

- 1. **Create the junction collection `daytrips_translations`:**
 - `id` (integer, auto-increment PK)
 - `daytrips_id` (uuid, FK → `daytrips`, ON DELETE SET NULL)
 - `translations_id` (uuid, FK → `translations`)
 - All translatable fields (e.g. `tour_title`, `teaser`, `tour_programm`)
- 2. **Create the M2M relation** linking `daytrips` to `translations` via the junction
- 3. **Add an alias field** on `daytrips` (type: alias, interface: translations or ai-translations)
- 4. **Content languages used:** de-DE (primary), de-CH, nl-NL

Example alias field config:

Field: `daytrips_description_translations`
Interface: `ai-translations` (custom AI Translations extension)


```
Related collection: daytrips_translations
Languages collection: translations
Language indicator field: code
```

The AI Translations interface adds the **AI Translate** button — editors can translate all fields from the source language to target languages with one click, review the purple-highlighted preview, and apply or cancel.

7.5 Creating Relations

M2O (Many-to-One):

```
"Add destination to daytrips as M2O to destinations, ON DELETE SET NULL."
```

Claude creates the integer field + the relation in one step.

O2M (One-to-Many):

```
"Add an alias field daytrips_routes to daytrips pointing to the daytrips_routes collection via daytrip_id."
```

M2M (Many-to-Many with junction):

```
"Create a M2M between daytrips and countries, junction table daytrips_countries, both FKs with ON DELETE CASCADE."
```

Claude creates the junction table, both FK fields, both relations, and the alias fields on each side.

7.6 Creating Automation Flows

Prompt pattern:

```
"Create a flow that triggers on daytrips create and update events, reads the MAT record, and writes the per-language mobility advice text into daytrips_translations.mobility_advice_text for all existing translation rows, creating missing rows if needed."
```

Claude will:

1. Read existing similar flows (e.g. the hotel MAT flows) for reference
2. Design the operation chain
3. Create the flow with the correct trigger
4. Create each operation in order, wiring them together
5. Verify the chain by reading the created flow back

Flow architecture patterns used in this project:

- **Event + sub-flow pattern:** CREATE and UPDATE event flows each fan out individual IDs to a TRIGGER sub-flow that does the heavy work. This prevents timeouts and allows parallel processing.
- **Parallel fan-out:** `iterationMode: "parallel"` on the trigger operation runs the sub-flow once per item ID simultaneously.

- **Permissions bypass:** Operations that write to readonly fields use `permissions: "$trigger"` and `emitEvents: false` to bypass UI readonly flags and prevent event loops.
 - **Schema-first:** AI always reads the schema of all involved collections before creating any flow.
-

8. Custom Extensions — How They Work with AI

8.1 What Extensions Are

Directus extensions are **compiled JavaScript bundles** that Directus loads at startup. Everything custom in the admin interface — the price tables, the AI translation button, the save-and-refresh button, the flow manager module, the schema management module — is a custom extension.

Extensions **are not installed** into Directus. They are **mounted into the Docker container** via a volume:

```
# docker-compose.yml
services:
  directus:
    volumes:
      - ./extensions:/directus/extensions
```

Directus scans the `extensions/` directory at startup and loads every extension it finds.

8.2 Extension Technology Stack

Layer	Technology
UI Components	Vue 3 Composition API (<code><script setup lang="ts"></code>)
Language	TypeScript
Build Toolchain	Directus Extensions SDK (Vite-based)
Native UI	Directus components (<code><v-button></code> , <code><v-icon></code> , <code><v-dialog></code> , etc.)
Theming	Directus CSS variables (<code>var(--theme--primary)</code> , etc.)

Build output per extension:

- `dist/app.js` — the compiled Vue component tree, loaded by the browser
- `dist/api.js` — the compiled server-side code (endpoints and hooks)

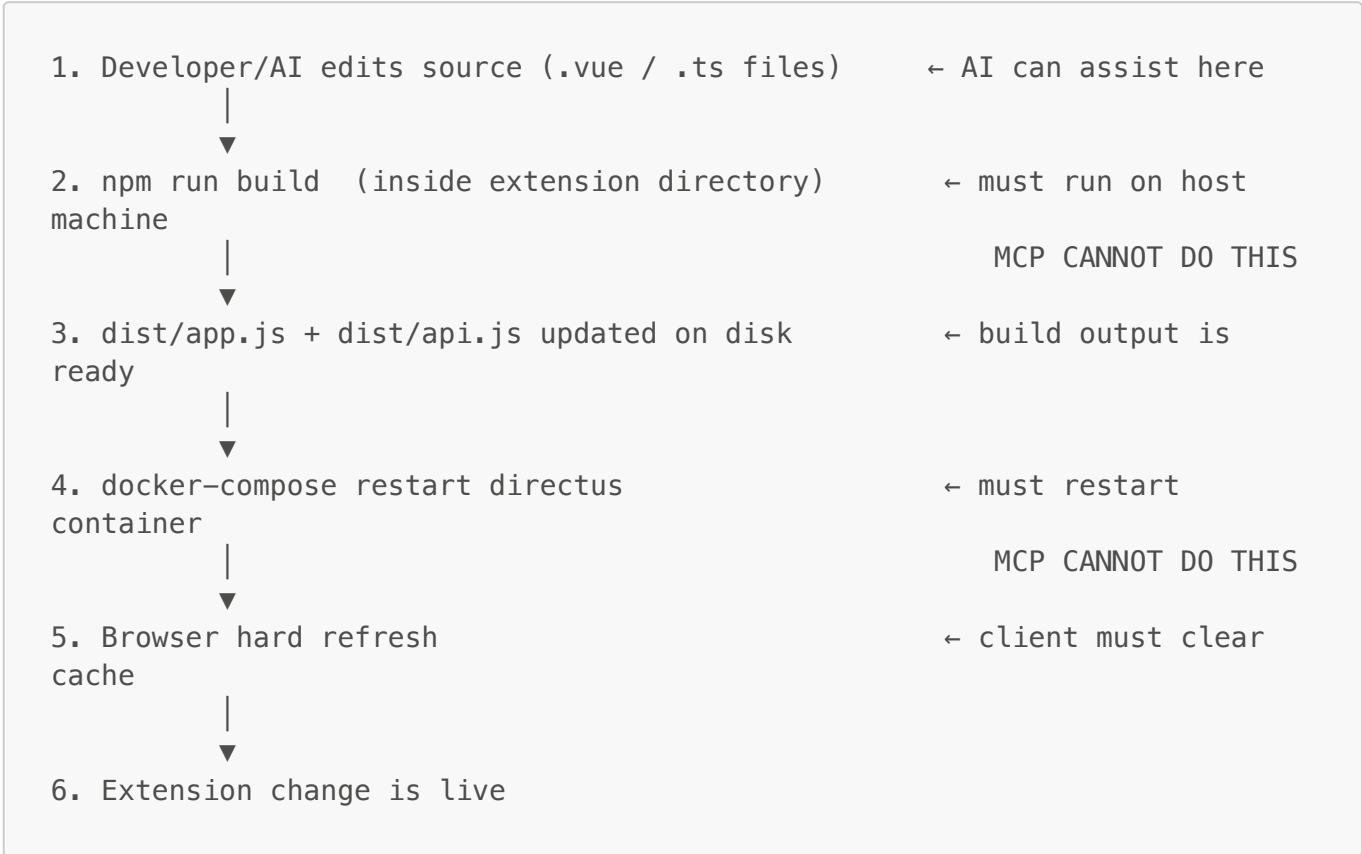
8.3 Extensions in This Project

Extension	Type	Purpose
<code>directus-extension-ai-translations</code>	Bundle (interface + endpoint)	AI Translate button in the translations view
<code>directus-extension-interface-room-prices-table</code>	Interface	Custom price matrix table for hotel rooms, cruises, and daytrips
<code>directus-extension-flow-manager</code>	Module	UI for managing and triggering flows
<code>directus-extension-save-and-refresh</code>	Interface	Save + auto-refresh button for complex records

Extension	Type	Purpose
<code>directus-extension-schema-management-module</code>	Module	UI for schema comparison and management

8.4 The Build and Deploy Cycle

AI can write, refactor, and fix extension code, but it cannot build or deploy them automatically. Every extension change requires:



8.5 Why MCP Cannot Update Extensions at Runtime

Reason 1 — MCP has no shell access. MCP tools operate entirely through the Directus REST API. They cannot execute shell commands (`npm run build`), write to the Docker container filesystem, or restart processes.

Reason 2 — Source edits have zero effect without a build. Directus only reads the compiled `dist/app.js`, never the TypeScript or Vue source files.

Reason 3 — Extensions are loaded once at startup. Even after replacing `dist/app.js` on disk, the running instance has already loaded the old version. A container restart is required.

Reason 4 — Browser caching. The admin app downloads compiled extension bundles once. Even after a server restart, the browser needs a hard refresh or cache bust to receive the updated bundle.

8.6 What AI Can and Cannot Do for Extensions

Task	AI + Claude Code	MCP
------	------------------	-----

Task	AI + Claude Code	MCP
Read and understand existing extension source	✓	✗
Write new Vue components / TypeScript logic	✓	✗
Refactor, fix bugs, add features to extensions	✓	✗
Design the data model an extension reads	✓ (with schema tool)	✓ (read-only)
Run <code>npm run build</code>	✓ (via shell in Claude Code)	✗
Restart the Docker container	✓ (via shell in Claude Code)	✗
Update <code>dist/app.js</code> through the Directus API	✗	✗
Hot-reload a running extension without restart	✗	✗

9. Critical Considerations Before Any Change — Directus Is a CMS, Not a Freeform App

This is the single most important section for anyone using AI (or working manually) on this project. **Directus is a Content Management System built on a relational database — it is not a custom application that can be freely rewritten.** Every collection, field, relation, and flow is bound by database rules and by dependencies on other objects. A change that looks isolated is rarely isolated.

For this reason, no change — whether made by AI via MCP or by a human in the admin UI — should ever be applied "blind." It must be preceded by analysis of the **current state**, the **intended new state**, and the **consequences and conflicts** between them.

9.1 Always Analyze Before You Change

Before creating, modifying, or deleting any collection, field, relation, or flow, the following analysis must be performed:

1. **Read the current state.** Use the schema tool to read the live structure of every collection, field, relation, and flow involved. Never assume a field name, type, or relationship — verify it against the live instance.
2. **Review what was changed previously.** Many objects in this project were built in phases (see the [STAGING_CHANGES/](#) logs and the Tours → Daytrips migration). A new change must be reconciled with prior changes, not applied on top of an outdated mental model.
3. **Define the intended new state precisely.** Know exactly which objects will be added, modified, or removed, and how they should look afterward.
4. **Identify conflicts between current and new state.** For example: a field name that already exists, a relation that already binds the collection, a flow already listening on the same event, or a value that violates an existing constraint.
5. **Map the consequences.** Determine what else depends on the object being changed — other collections, relations, flows, interfaces, extensions, and existing content rows.
6. **Propose, then confirm.** AI proposes the full plan with consequences spelled out; a human approves before anything is written. Destructive operations are always confirmed individually.

9.2 Why a Change Is Rarely Isolated — Database & Content Consequences

Because Directus sits directly on a relational database, schema changes have real, sometimes irreversible, effects on both structure and stored content:

Change	Possible consequence on DB / content
Deleting a field	Permanently drops that column and all data stored in it across every row. Cannot be undone without a backup.
Changing a field type	May fail, truncate, or corrupt existing values if the data is not compatible with the new type (e.g. text → integer).
Renaming a field	Breaks any flow, interface, extension, display template, or API consumer that references the old name.

Change	Possible consequence on DB / content
Deleting a collection	Drops the table and, depending on FK rules, cascades deletes or nullifies related rows in other collections.
Changing a relation's ON DELETE rule	Alters what happens to child rows when a parent is deleted (CASCADE deletes them, SET NULL orphans them). Wrong choice causes silent data loss or orphaned records.
Adding a required field to a collection with existing rows	Existing rows may become invalid or block saves until backfilled.
Adding/editing a flow trigger	Can fire on existing automation events, create loops, or perform mass writes the moment it is saved.
Editing translation junctions	Can detach or duplicate per-language content rows if the junction or FK is misconfigured.

The key principle: **content and schema live in the same database**. You cannot change the schema without considering the content already stored against it, and you cannot bulk-edit content without considering the flows and constraints that react to it.

9.3 Ripple Effects — One Change Can Affect Many Others

This project is highly interconnected. Products (hotels, cruises, tours, daytrips) share patterns: translation tables, price-calculation flow chains, margin/exchange presets, image-badge flows, and the shared MAT (Mobility Advice Text) system. A change in one place can ripple outward:

- **Shared collections:** `translations`, `destinations`, `rates`, `margin_presets`, `exchange_rate_presets`, and `mobility_advice_text` are referenced by many products. Editing one of these affects every product that depends on it.
- **Flow chains:** Price calculation is a chain of flows (trigger on price date → trigger on category → trigger on occupancy → sync prices). Changing one link can break or skew downstream calculations.
- **Junction and alias fields:** Adding or removing a relation often requires matching alias fields, junction tables, and `one_field` settings on the inverse side — miss one and the UI or API breaks.
- **Presets feeding multiple products:** `margin_presets` and `exchange_rate_presets` push values into hotels, cruises, tours, and daytrips simultaneously. A preset change recalculates many products at once.
- **MAT and translation sync:** Editing the single `mobility_advice_text` record re-syncs text into every product's per-language rows via flows.

Before any change, ask: what else reads from or writes to this object? AI must trace these dependencies (using the schema and flows tools) and surface them as part of the proposal, so the human understands the full blast radius before approving.

9.4 The Rules Directus Enforces

Directus is opinionated and enforces conventions that cannot be bypassed:

- **Snake_case, plural collections, singular/plural field naming** (see Section 7.1) — deviating breaks consistency and tooling.
- **Translations require a junction table and an alias field** — you cannot just "add a translatable field."
- **Relations must define both sides** (the FK and, usually, the inverse alias / `one_field`).
- **Primary key types are fixed at creation** (UUID vs auto-increment integer) and differ across this project's collections (e.g. tours use integer PKs, hotels/cruises use UUID).
- **System collections** (`directus_users`, `directus_roles`, `directus_permissions`, `directus_settings`) are managed by Directus and are intentionally not exposed to MCP.
- **Readonly fields** populated by flows must be written with the correct permission/event-bypass settings, or the write is rejected or triggers loops.

Working *with* these rules — not against them — is what keeps the instance stable. This is precisely why AI is instructed to read the schema first and follow existing patterns rather than invent new structures.

10. Migration Management — Environments & Deployments

10.1 Environment Strategy

BoTG:

- Development → Staging → Production
- All schema and collection changes must start in Development or Staging
- Never make direct schema changes in Production

Recommended workflow:

1. Make all schema, collection, field, and flow changes in Staging
2. Perform thorough testing and validation in Staging
3. Deploy approved changes to Production only after verification

10.2 Why Directus Has No Native "Push to Production"

Directus does not provide a native "Push to Production" button or mechanism. This is a fundamental architectural limitation: each environment has its **own separate database**, and Directus has no built-in mechanism to selectively migrate individual changes between them.

The core challenges:

Challenge	Explanation
ID and UUID mismatch	The same record has different internal IDs in staging vs production
No change tracking	Directus does not track which changes were made since the last migration
Content conflicts	The same record may have been modified in both environments
Object dependency order	Collections, fields, relations, flows must be migrated in the correct dependency order
Selective migration	There is no way to choose "migrate only these 3 new fields" without a custom tool
Media assets	Files and folders have separate UUIDs and storage paths per environment
Roles and permissions	User roles and permission sets are not automatically synchronized

10.3 What a Custom Migration Tool Would Require

To build a proper "Push to Production" solution, the following would need to be designed and developed:

1. **Change detection** — compare schema, flows, content, and permissions between environments before deployment
2. **Conflict presentation** — show the user what is different and what would be overwritten
3. **Selective migration** — allow the user to choose what should and should not be migrated
4. **Conflict resolution** — handle cases where the same object was changed in both environments
5. **Rollback mechanism** — ability to undo a migration

- 6. **Object type coverage** — define which types of objects are included: collections, fields, flows, permissions, content, media, translations, settings
- 7. **Hosting and access** — decide where and how the tool runs (Directus extension, external application, CLI script)

This would be a **custom-built solution** with significant development effort. Scope and cost would need to be estimated based on the desired coverage.

10.4 Our Current Migration Approach

In the absence of a native migration tool, we follow a structured manual process:

For schema changes (collections, fields, relations):

- 1. All changes are made in Staging via AI + MCP
- 2. Every change is logged in a **STAGING_CHANGES/** directory as a detailed markdown file
- 3. Each log includes: what was created, field-by-field details, relation definitions, flow UUIDs, and a **Revert Procedure** with step-by-step rollback instructions
- 4. Once verified in Staging, the same operations are repeated in Production via AI + MCP
- 5. The AI reads the staging change log and replicates the exact same schema in production

For flow changes:

- Flows are created/modified in Staging via AI + MCP
- The complete flow structure (operations chain, trigger config, all operation UUIDs) is documented
- Replicated to Production in the same session or from the documentation

For content/data changes:

- Content is generally not migrated between environments
- Content created in Staging stays in Staging; Production content is entered directly in Production

10.5 Real Example — Tours → Daytrips Migration

The most complex migration performed in this project was the **Tours → Daytrips migration**, completed in Staging in June 2026. This is a concrete example of how AI + MCP handles large schema changes.

Strategy: Create all new **daytrips_*** infrastructure in Staging **without touching any existing tours_* collections**. Only after full verification would the old tours structure be cleaned up.

10 phases, all executed via AI + MCP:

Phase	What Was Created
1	30+ new fields on daytrips and daytrips_categories
2	8 new translation and pricing collections (daytrips_translations , daytrips_translations_1 , daytrips_image_badge_translations , etc.)
3	5 M2M junction tables (daytrips_countries , daytrips_directus_files , etc.)
3b	2 pricing collections (daytrips_prices , daytrips_prices_translations)

Phase	What Was Created
4	Batch pricing system (batch_daytrips , batch_daytrips_locale)
5	ERP/MP exception tables (4 new collections)
6	16 alias fields on daytrips
7	18 automation flows (Image Badge, MAT, Price Calculator, Surcharges, Margin, Exchange Rate)
8	Preset translation tables + alias fields on margin_presets and exchange_rate_presets
9	MAT system for daytrips (new field + 3 new flows)
10	Data migration: 3 daytrips records created with all related data

Each phase was documented with exact field definitions, relation configurations, and revert procedures.

10.6 Snapshots and Backups

Directus does not provide a built-in snapshot and restore system. A complete backup/restore solution would require a custom implementation defining:

- What to include: content, schema, flows, permissions, media
- Backup frequency and retention period
- Full vs selective restore
- Conflict handling for changes made after the backup

In the current setup, database-level backups are the primary safety net. For schema, the [STAGING_CHANGES/](#) documentation provides a manual revert path via AI + MCP.



11. Collaboration, Content Safety & Deployment — Policy and Assessment

This section sets out our assessment and recommended practices for collaboration, content safety, environment management, and deployment across the **BoTG environments**.

11.1 Where Content Can Be Edited So That It Is Never Lost

The available BoTG environments are:

- **BoTG:** Development, Staging, Production

It is important to note that Directus does **not** provide a native mechanism to isolate individual content, schema, flow, or configuration changes during environment migrations.

Since each environment has its own database, migrations operate at the database level. During deployment, it is not possible to reliably identify and selectively preserve only certain content, collections, fields, flows, permissions, or other changes that were manually created in another environment.

To achieve selective, conflict-aware migrations, we would need to develop a custom comparison and migration solution that:

- Detects differences between environments before deployment
- Presents those differences to the user
- Allows selection of what should and should not be migrated
- Handles conflicts where the same records or structures have been modified in multiple environments

Additional considerations include:

- Where and how such a tool would be hosted and maintained
- When the comparison should be executed
- How to handle manual changes made directly in Production
- How to resolve conflicting changes between environments
- Which types of objects should be compared (content, collections, fields, flows, permissions, translations, etc.)

Depending on the desired scope and level of automation, we can evaluate whether a custom application or migration utility would be the most appropriate approach and estimate the required effort accordingly.

11.2 Where Collections Should Be Edited

Collections, fields, relations, and other schema changes can be managed through the **Data Model** section within Directus Settings.

To maintain a controlled workflow, we recommend:

- Making all schema and collection changes in **Staging** (or Development for BoTG)
- Performing thorough testing and validation
- Deploying approved changes to **Production** only after verification

This approach applies across all BoTG environments and helps reduce the risk of unintended production changes. AI + MCP is our primary tool for applying schema changes — the AI reads the staging environment, applies the changes, and documents everything with revert instructions before production is touched.

11.3 Pushing Changes from Staging to Production

Directus does **not** provide a native "Push to Production" functionality.

While it is technically possible to build a custom deployment tool, there are numerous challenges and considerations:

- Determining which objects should be migrated (collections, fields, flows, permissions, content, media, etc.)
- Ensuring Staging and Production are sufficiently aligned before deployment
- Managing differences in IDs, UUIDs, and references between environments
- Handling content conflicts and overwritten changes
- Managing user roles, permissions, and policies
- Establishing secure communication between environments (SSH, APIs, or other methods)
- Defining rollback procedures in case of deployment issues

Because of these complexities, such functionality would require a custom solution, either as:

- A dedicated Directus extension
- An independent deployment application
- An enhancement of the existing BoTG migration tooling

The exact effort would depend on the scope of deployment functionality required.

Current workaround: We use AI + MCP to replicate schema changes from staging to production. The AI reads the staging change log (our [STAGING_CHANGES/](#) documentation), then executes the exact same MCP tool calls against the production connection. This is manual but accurate, auditable, and reversible.

11.4 Snapshots / Backups with Revert Functionality

Directus does **not** provide a complete environment snapshot and restore system out of the box.

A custom solution would need to define:

- What should be included in backups: content, collections, fields, flows, permissions, media assets
- How frequently backups should be created
- How long backups should be retained
- Whether restores should be full or selective
- How to handle changes made after a backup was taken

One of the major challenges is **conflict management**:

- Important changes may have been made after the backup was created
- A full restore could unintentionally overwrite newer content

- Selective restores require object-level comparison and reconciliation

Similar to environment migration and push-to-production (Sections 11.1 and 11.3), implementing a robust snapshot and rollback mechanism would require a custom extension or application, potentially integrated into the existing migration tooling.

Current fallback: Database-level backups (at the Docker/server level) provide a last-resort restore point. For schema, the [STAGING_CHANGES/](#) documentation provides step-by-step revert procedures for every change made via AI + MCP.

11.5 Enabling Directus MCP / Working with Claude

MCP can be enabled and used with Claude or other MCP-compatible AI tools, provided the Directus version and environment meet the necessary requirements.

Minimum required version: Directus **v11.12.0** or higher.

Current status:

- **BoTG (all environments):** Running v11.17.4 — MCP is supported and can be enabled immediately

To enable MCP (per environment where MCP should be available):

1. Navigate to [/admin/settings/ai](#)
2. Enable the MCP server
3. Open the desired user account
4. Generate an MCP access token for that user
5. Copy and securely store the generated token

To connect Claude: See Section 4 of this document for full instructions (OAuth and static token methods).

Please let us know if you would like us to assist with the MCP configuration.

12. What AI Cannot Do — Limitations

Understanding limitations is as important as understanding capabilities. This section covers both hard technical limitations and reliability boundaries.

12.1 What MCP Cannot Do

Limitation	Explanation
Upload binary files	The <code>files</code> tool can only import from a URL or update metadata. It cannot accept a file upload directly from the AI session. Files must be uploaded through the Directus UI or REST API.
Access to Directus UI	MCP interacts with the API layer only. It cannot see, click, or screenshot the admin interface.
User management (create/delete users)	The MCP tools do not expose <code>directus_users</code> CRUD — intentionally kept out for security.
Role and permission editing	<code>directus_roles</code> and <code>directus_permissions</code> are system-level resources not exposed by MCP.
Extension code execution	AI can read and write extension source code files on the local filesystem, but cannot install, build, or hot-reload extensions through MCP — that requires shell access and a build process.
Raw database migrations / SQL	Raw SQL is not accessible. Schema changes go through the <code>collections/fields/relations</code> tools, which use Directus's own migration system.
Webhooks configuration	<code>directus_webhooks</code> for direct management is not exposed.
Global settings	<code>directus_settings</code> (project name, logo, auth settings) is not exposed.
Dashboard/Panel creation	Insights dashboards and panels are not manageable via MCP.
Real-time subscriptions	MCP is request-response based; it cannot subscribe to live data changes.
Build or restart Docker containers	MCP has no shell access and cannot trigger builds, restarts, or deployments.

12.2 What AI Cannot Do Reliably (Even With MCP)

Limitation	Explanation
------------	-------------

Limitation	Explanation
Guarantee correctness without schema check	AI should always read the schema before writing. If it skips this step, it risks guessing field names incorrectly. Our workflow enforces schema-first.
Handle high-volume bulk operations safely	Modifying thousands of records in a single instruction is risky. Large bulk operations should be reviewed, batched, and confirmed by a human before execution.
Make business logic decisions	AI can implement logic it is told, but cannot decide what the right pricing rule is, which destination should have which margin, or what the brand voice should be. Those decisions stay with the business.
Recover from bad deletes	If a delete is confirmed and executed, MCP cannot undo it. Directus has revisions, but reconstruction from revisions requires manual steps. We always confirm before deleting.
Work offline or in air-gapped environments	Claude requires an internet connection to the Anthropic API. MCP connections to Directus are local, but the AI itself is cloud-based.
Read encrypted/hashed field values	Fields with type hash (passwords) are one-way and can never be read back — not by AI, not by anyone.
Guarantee translation quality	AI translation produces drafts. Human review is required for market-facing copy, legal text, and brand-sensitive content.
Replace human judgement on content	AI can draft, suggest, fill, and organize content, but editorial decisions, brand voice validation, and legal accuracy must be verified by humans.
Guarantee no impact on production	Even with staging-first workflows, unexpected data states in production can cause issues. Always have a revert plan before applying changes to production.

13. Summary Capability Table

Capability	MCP (with Claude)	AI (without MCP)
Read live database content	✓	✗
Understand schema automatically	✓	✗ (must be described manually)
Create/update content items	✓	✗
Create collections and fields	✓	✗
Create and manage flows	✓	✗
Trigger existing flows	✓	✗
Manage file metadata	✓	✗
Analyze image content visually	✓ (via assets tool)	✗
Upload binary files	✗	✗
Manage users/roles/permissions	✗	✗
Build or restart Docker containers	✗	✗
Read/write extension source code	✗	✓ (via Claude Code filesystem)
Run <code>npm run build</code> for extensions	✗	✓ (via Claude Code shell)
Make editorial/business decisions	✗	Advisory only
Guarantee translation quality	✗	Draft quality only
Access Directus UI directly	✗	✗
Native push staging → production	✗	✗ (requires custom tool)
Native backup/restore	✗	✗ (requires custom tool)

14. Security Best Practices

14.1 Token and Access Security

Practice	Why
Create a dedicated AI user (e.g. <code>ai-agent@company.com</code>)	Audit trail shows "ai-agent" did it, not a human user
Assign minimum required permissions via a Role	AI can't accidentally delete things it shouldn't touch
Never commit the token to Git	Use environment variables or a secrets manager
Use different tokens per environment	Staging breach doesn't affect production
Store tokens in <code>.env</code> or a secrets vault	Not hardcoded in config files
Rotate tokens if an environment is compromised	Revoke immediately from the user record in Settings → Users

14.2 Example Role Permissions for a Read-Heavy AI Agent

Collections:	Read (all)
Files:	Read, Update (metadata only)
Flows:	Read, Trigger (manual only)
Schema:	Read only
Users:	No access
Settings:	No access

For a write-enabled AI agent (used in development/staging schema work):

Collections:	Read, Create, Update, Delete
Fields:	Read, Create, Update, Delete
Relations:	Read, Create, Update, Delete
Items:	Read, Create, Update, Delete (per collection, restricted)
Flows:	Read, Create, Update, Delete, Trigger
Files:	Read, Create, Update
Schema:	Read
Users:	No access
Settings:	No access

14.3 What to Always Confirm Before Executing

These operations should always be reviewed and approved by a human before the AI executes them:

- **Delete any item, collection, field, relation, or flow**
- **Bulk update of more than 50 records**
- **Any operation on a Production environment**

- **Changes to pricing or sell price fields**
- **Changes to flow trigger conditions**
- **Changes to permissions or role configurations**

14.4 Audit Trail

Because we use a dedicated AI user for MCP operations, every change made by the AI appears in Directus's activity log under that user account. This provides a clear audit trail distinguishing human edits from AI-assisted edits.

Appendix A — Directus MCP Tool Reference

Tool	Purpose
schema	Read collection and field definitions
items	CRUD on any collection
collections	Create, read, update, delete collections
fields	Create, read, update, delete fields
relations	Create, read, update, delete relations
files	Read and manage file metadata
assets	Retrieve file content (images, audio) as base64
folders	Create and manage virtual folders
flows	Read, create, update, delete flows
operations	Read, create, update, delete flow operations
trigger-flow	Execute a manual flow with a payload
system-prompt	Returns project-specific AI instructions

Appendix B — Useful Directus AI Documentation Links

- MCP Overview: <https://directus.com/docs/guides/ai/mcp>
- Supported Clients: <https://directus.com/docs/guides/ai/mcp#supported-clients>
- Installation & Connection: <https://directus.com/docs/guides/ai/mcp/installation#connect-your-ai-client>

This document is maintained by Encircle Technologies. For questions or updates, contact the project team.